0. Orientation: What Linux Is and How to Practice

00. Orientation: What Linux Is and How to Practice

Phase: Phase 0 — Orientation
Estimated time: 1 day
Slug: `00-orientation`

Learning objectives

- Explain Linux vs Unix vs GNU userland at a beginner level
- Choose a practice environment (VM, cloud, WSL, or container)
- Open a shell and run your first commands safely
- Adopt the study method: attempt → struggle → solution → rewrite

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Linux runs most of the servers, containers, cloud instances, routers and Android phones in the world. If you work in backend, DevOps, security or data engineering, you will meet a Linux shell sooner or later, often on a machine that has no graphical desktop at all. Learning it once pays off on every distribution you touch.

This course is practical. You will type commands, break things in a safe sandbox, and read error messages until they stop being scary. Reading output carefully is the real skill; memorizing flags is secondary because `man` and `--help` are always available.

Core concepts

“Linux” strictly means the kernel: the program that manages CPU, memory, devices and processes. A *distribution* (Ubuntu, Debian, Fedora, Alpine, Arch) bundles the kernel with a C library, an init system, package manager and userland tools (mostly from the GNU project, which is why you also hear “GNU/Linux”).

Term	Meaning	Example
Kernel	Core that talks to hardware	`uname -r` shows its version
Distribution	Kernel + packages + defaults	Ubuntu 24.04
Shell	Program that reads your commands	`bash`, `zsh`, `sh`
Terminal	Window that displays a text session	GNOME Terminal, tmux pane
Package manager	Installs software	`apt`, `dnf`, `apk`

Terminal and shell are different things: the terminal draws text and sends keystrokes, the shell interprets them and starts programs.

```
you --keys--> [ terminal ] --> [ shell: bash ] --fork/exec--> [ program ]
                                      |                           |
                                      |                     system calls
                                      v                           v
                                  [ libc ]  ------------------>  [ kernel ] -->
hardware
```

Everything you run ends up as system calls into the kernel. A useful motto: in Unix, most things are files or processes.

Safe places to practice: a virtual machine, a Docker container, WSL2 on Windows, or a cloud VM you can delete. Avoid experimenting on a machine holding data you care about.

Worked examples

Find out where you are and what you are running:

```
uname -sr
# sample output: Linux 6.8.0-45-generic
cat /etc/os-release | head -n 2
# sample output: PRETTY_NAME="Ubuntu 24.04.1 LTS"
# sample output: NAME="Ubuntu"
```

`uname -sr` prints kernel name and release; `/etc/os-release` describes the distribution on nearly every modern system.

Who and where:

```
whoami
# sample output: student
pwd
# sample output: /home/student
echo "$SHELL"
# sample output: /bin/bash
```

`whoami` prints your username, `pwd` prints the current directory, and `$SHELL` holds your login shell (it may differ from the shell currently running).

Getting help offline:

```
man ls | head -n 3
# sample output: LS(1)                User Commands               LS(1)
ls --help | head -n 1
# sample output: Usage: ls [OPTION]... [FILE]...
```

`man` opens the manual page; press `q` to leave. `--help` gives a shorter summary for most GNU tools. Builtins such as `cd` are documented with `help cd`.

A tiny greeting, the shape of the exercise:

```
name="Ada"
echo "Hello, $name!"
# sample output: Hello, Ada!
```

Mini-lab

1. Start a throwaway environment: `docker run --rm -it ubuntu:24.04 bash` (or open any Ubuntu VM or WSL shell).
2. Run `uname -a` and `cat /etc/os-release`; write down kernel and distribution.
3. Run `whoami`, `pwd`, `hostname`. In a container you are usually `root`.
4. Run `echo $0` to see which shell you are in.
5. Run `man man` and read the first screen, then press `q`.
6. Create a file: `echo "hello linux" > ~/hello.txt` then `cat ~/hello.txt`.
7. Run `ls -l ~/hello.txt` and identify the size and owner.
8. Exit with `exit` (or Ctrl-D). The container disappears because of `--rm`.

Decision guide / failure modes

Use a VM when you need a full OS with its own kernel and services; use a container for fast, disposable userland (it shares the host kernel); use WSL2 for daily work on Windows; use a real install only when you need hardware access or want to commit.

Common failures:

- `command not found`: typo, tool not installed, or not in `$PATH`. Check with `type NAME` and install via the package manager.
- `Permission denied`: you lack rights on that file. Do not reflexively add `sudo`; read the error first.
- Copy-pasting commands with `rm -rf` or `curl | sh` from the web without reading them. Read first, then run.
- Confusing case: Linux file names are case-sensitive, so `Notes.txt` and `notes.txt` are different.

- Practicing as root in a container, then being surprised on a real system where you are not root.

Common mistakes

- Practicing only on macOS and skipping Linux-specific topics forever
- Running destructive commands as root without understanding them
- Skipping the rewrite-from-memory step

Exercises

Work in `exercises/00-orientation/`. Read that folder's `README.md` fully before coding.

```
cd exercises/00-orientation
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement greet(name). It takes a single string NAME and returns the greeting text "Hello, NAME!" exactly, with a comma, one space and an exclamation mark at the end. Do not print extra whitespace or a trailing newline inside the returned value. If NAME is an empty string, return "Hello, world!" instead. Keep the function pure: no input from the keyboard, no file access. This mirrors what a shell function would do with echo "Hello, $1!" and warms you up for editing and running scripts.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/00-orientation/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is the kernel vs userland?
2. Name two practice environments suitable for this course
3. Why avoid looking at solutions first?

Further reading

- https://www.kernel.org/
- https://ubuntu.com/tutorials/command-line-for-beginners